1. 进入运动控制任务 00:00-00:49

本节先来讲第一个任务，这个任务需要根据不同的条件来进行创建，如果是用户按键被按下的情况进行复位或者上电的话，创建的是辅助自检模式，相反如果复位或者上电的时候用户按键没有被按下的话，创建的就是运动控制任务。

那本节要来介绍的就是运动控制任务，辅助自检任务在后续也会做介绍。

2. 绝对延时及计时 00:49-03:15

那看到运动控制任务的内容，进来之后首先是读取了滴答定时器的数值，读取这个数值是为了下面做绝对延时的时候，作为绝对延时函数的参数，那这里是只需要读取一次就可以的了，因为传进去的这个参数是这个变量的地址，在绝对延时函数里面会更新这个值，那当前的这个绝对延时函数是为了控制这个任务以 100Hz 的频率来执行。

往下的话是有一个小于 3000 的计数，也就是在刚开机的前 30s 都会进行计数，30s 之后停止计数，这里主要是作用在刚开机的前段时间陀螺仪的零点读取、蜂鸣器的提醒等等，且在开机的前 10s，陀螺仪数据还在读取零点，所以像 APP 控制、ROS 控制、PS2 手柄控制等等各种控制模式不能进行，这些地方都需要用到这个计数值，具体各种控制模式在后续也会做详细的介绍。

3. 读取编码器并转换成实时速度 03:15-11:30

接下来就是读取编码器的数值了，Get_Velocity_Form_Encoder()这个函数的主要作用就是读取编码器的数值，并把读取到的数据结合前面提到的小车的一些参数转换成以 m/s 为单位的速度。

进入到这个函数可以看到首先是读取了四个电机的编码器数值，因为电机的安装或者是机械结构上难免出现一些误差，所以读取了四个电机的编码器数值之后需要计算一下纠偏系数，我们可以通过这个纠偏函数来调整小车走直线的效果，其实主要调整的就是 LineDiffParam 这个变量，当前这个变量是 50，在这个函数里面，对于左轮的纠偏，50 ~ 100 对应的就是 1.0 ~ 1.2 的纠偏系数，对于右轮的纠偏，50 ~ 0 对应的也是 1.0 ~ 1.2，当前 50 其实相当于没有做任何的纠偏，如果小车拿到手上看小车走直线的效果不理想的话，可以通

过修改这个参数去修改走直线的效果。

获取编码器数值之后需要来根据小车的车型来决定这个编码器数值的极性，代码中是分成了三组，第一组是麦轮小车还有四轮小车，这一组的特点都是四个轮子，以麦轮小车来看，先打开“轮式移动机器人的运动学模型”（**在资料包中的“通用资料 -> 2.运动底盘的控制与运动学解析”**），找到麦轮小车的运动学模型，假如说要让四个轮子都往前转的话，对于左边的两个轮子来说，对应电机实际上都是逆时针转动的，但是对于右边两个轮子来说，电机都是顺时针转动的，这里就可以知道了两边的轮子有一个对称的关系，也就是两边极性的相反的。那看回到代码，首先是给了 A 电机的编码器变量赋了原本的值，这样的话 B 电机也应该是原值，而 C 跟 D 的话就是相反的，所以多加了一个负号。接下来是阿克曼小车、差速小车以及履带小车，这两个小车的特性就是均使用了两个电机，根据前面对麦轮小车的分析可以知道，小车左右两边的电机极性是相反的，所以对于两个轮子的小车，左电机给了原值， 右电机就是相反的，后面的 CD 电机实际上的没有使用到的，不管加不加负号，对实际上的使用都没有影响。最后是全向轮小车，如果要让全向轮小车的轮子均往同一个方向转动，可以看到电机都是往一个方向转动的，且小车逆时针转动的时候是 Z 轴的正方向，这个时候电机是顺时针转动的，按前面对其他车型的分析可知，顺时针转动都要加上一个负号，所以 ABC 电机都要加上负号，D 电机因为没使用到，所以加不加负号对小车的可知并不影响。

处理好编码器的极性问题之后，就需要把这个编码器数值转换成车轮的速度，以 A 电机为例，读取到的编码器数值除以电机转动一圈能读到的数值得到的就是电机转动多少圈，那这个电机转动一圈能读到的数值就是电机减速比*编码器精度*倍频数 = 13（假如是霍尔编码器）*30*4 = 1560，得到了圈数之后，因为这个数值是每 10ms 读取一次的数值（任务执行频率是 100Hz，也就是 10ms 执行一次），所以转换成每秒电机转动的圈数就需要再乘以读取频率，最后转换得到当前速度每秒可以转动多少圈，这个时候要转换成 m/s 的话，只需要知道车轮转动一圈是多少 m 即可，也就是计算车轮的周长，再乘以前面得到的数值即可。后面的 BCD 电机也是同样的计算方法。

计算完还要对前面的纠偏系数做具体的使用。

4. 按键功能 11:30-12:42

获取了实时速度之后，再往下是读取按键的函数，在运动可知任务里面，按键有多个功能，如果是单击的话，则是开启自动回充的功能，当然前提是有自动回充的设备；双击的话则是更新陀螺仪的零点，因为陀螺仪是存在温漂零漂的，所以小车在运动了一段时间后可能

会出现陀螺仪数据漂移的情况，可以退通过双击来更新零点；长按的话则是切换到显示屏的另一个页面。

5. 自动回充 12:42-13:17

按键下面则是自动回充的控制代码，如果有购买我们的自动回充设备的话，则可以通过这部分代码实现自动回充的功能，如果没有购买的话这部分就不需要去了解了，那具体这部分代码任何实现的呢，可以看一下自动回充的资料包，如果原本提供的资料包中没有这个资料的话，可以找后台客服要一下即可。

6. 控制模式 13:17-17:23

自动回充的下面是各种模式的控制，也就是小车如果不是执行自动回充的话，就进入到这些控制模式来，那这几个模式如何来切换呢？首先开机默认是 ROS 控制模式，如果想要切换成 APP 控制模式的话，那我们可以直接使用手机 APP 连接蓝牙，然后通过 APP 直接向单片机发送 5 个 A，这个时候就会切换为 APP 控制模式，如果是航模或者 PS2 手柄的话，也是直接连接之后执行一个特定的动作即可进入到对应的模式，当然我们可以从板子上的 OLED 显示屏的左下角看到当前的控制模式，那这些模式的话后续我们还会做更详细的介绍，本节就不细讲了。

那不管是 APP 控制、航模控制或者是 PS2 手柄的控制，实际上最后都是转换成 XYZ 三轴的速度之后，将这三个变量给到 Drive_Motor 函数做运动学逆解得到各个电机的目标速度，而其他的像 ROS 控制、CAN 总线控制等则是直接接收到 XYZ 三轴的速度，同样的也是给到 Drive_Motor 函数做运动学逆解。

7. 运动学分析 17:23-26:34

那这个运动学逆解的函数，进来之后可以看到目标速度最高是限制在了 3.5m/s，再对传进来的 XYZ 三轴的速度做限幅，那因为全向轮小车的 ROS 控制里面没有速度平滑处理的功能包，所以在 STM32 这边会给这几种车型增加这个功能包，也就是下面的平滑处理。那这个平滑处理的函数实际上就是对这几个速度做一个匀速递增或递减的过程，进入函数体可以看到，对于对于 XYZ 轴都是加或者减 0.01，直到达到目标速度的，如果目标速度都是 0 的话，则是乘以 0.9，让速度慢慢降下来直到 0，最后再做个限幅，避免加或减的时候超出原本设定的数值。

平滑处理之后还需要计算一下纠偏系数，那前面纠偏系数的作用已经介绍了这里就不多讲了。

往下则是运动学逆解的过程了，运动学逆解其实就是利用 XYZ 这三轴的速度配合上小车机械机构相关的一些参数计算出具体电机的目标速度，可以看到麦轮小车的运动学逆解也是通过 XYZ 三轴的速度计算得到目标的电机速度，那具体关于运动学逆解的解析的话可以参考我们资料里面对各种车型的运动学分析的资料，计算出具体的目标速度之后再对计算的速度做一个限幅，然后再成乘以纠偏系数。那全向轮小车也是一样的分析过程，因为 D 电机没有使用到，所以代码中直接给了 0。然后是阿克曼小车，阿克曼小车会特殊一点，因为他的机械结构上是有一个舵机来控制前面两个轮子的转向角的，所以这个车型会多了一个舵机的控制，那对于阿克曼小车来说，Z 轴代表的就是前轮转动的角度，有了这个角度则可以算出阿克曼小车在转弯时的半径，**那这里需要说明一下，如果是串口跟 CAN 总线这些控制的话，接收到的控制命令是车体的角速度，然后再去转换成前轮的转角的，而像 APP 控制、航模控制以及 PS2 手柄控制这些模式都是直接设置前轮转角的，**那在运动学模型的手册中不难看出，阿克曼小车转动的时候，他的中心点是在车身外面 的，所以不想麦轮小车一样可以在原地转东，只能以车身外的某个点来转动，也就是转动的时候存在这个半径，那我们前面计算的这个半径之后还需要做限幅，避免舵机转动的角度过大，那在运动学逆解的过程中，就比其他车型还多了一个舵机 PWM 值的计算，最后 CD 两个电机因为没使用到所以直接赋值 0 即可，同样的，具体的运动学逆解过程可以参考资料包中的内容做更详细的了解。然后是差速小车，因为跟阿克曼小车的运动学逆解很像，就是少了个舵机而已，这里就不多讲了。四驱车跟前面的麦轮小车也是类似的。最后是履带车，实际上跟差速小车也差不多，那这里也不多讲，可自行触类旁通即可。

8. PID 控制 26:34-31:38

获取完电机的目标速度之后，我们就有了小车的各个电机的当前速度以及目标速度了，这个时候就可以来对小车进行控制了，但是在控制小车之前，需要判断当前电池电压是否正常、使能开关是否在 ON 档位、软件的停止位是否为 0，或者如果是使用了自动回充的话，那自动回充的标志位是否为 1、使能开关是否在 ON 档位、软件停止位是否为 0，如果满足这两个条件中的其中一个的话，那么久可以进入到 PID 控制这里面来了。

以电机 A 为例来看一下 PID 控制的函数，先看到代码中对函数体的注释说明，这个函数是一个增量式的离散 PID，公式也列出来了，需要用到的参数就是本次的偏差以及上一次的

偏差，那正常在速度控制闭环系统里面是只是用 PI 控制的，所以 D 项就去掉了，公式也更简单了，那关于 PID 公式的推到或者是 PID 相关内容的详细的话，也可以看看资料包中直流电机基础部分的内容，里面也有 PID 相关的学习。

那在函数体中可以看到，进来是先定义了几个变量，都是用 static 做修饰的，所以要使用四个电机的话，是不能使用同一个函数的。接下来则计算前面提到的本次误差跟上一次的误差，本次误差则是目标速度减去当前速度，接下来则套用 PI 控制的公式来计算结果，最后在退出这个函数之前需要记录下当前误差，因为是 static 修饰的静态变量，所以这个变量在函数结束后不会被释放，下一次进入这个函数的时候，这个变量的值就是上一次的误差了。那可以看到在记录上次误差之前还会对 PWM 数值做一个限幅，这里的限幅是在 16700，因为 PWM 信号是由定时器提供，这里可以看到初始化的时候，满 PWM 的话是 16799，所以这里限幅在 16700 也是很合理的。

那前面是普通的 PID 计算过程，在这个函数结束之前可以看到还有清楚 PID 的部分，这一部分主要作用就是在小车停下来的时候可以让 PWM 信号逐渐减到 0，这样的话可以减少电机受到不确定信号的影响而造成的轻微移动的问题，同时小车在停下的时候也不会因为 PWM 数值的累积而产生电流声。

通过 PID 计算得到目标 PWM 数值之后，还需要对这个数值进行限幅，限幅完成之后则是对是否需要清除 PID 做检测，如果需要的话则自动执行，这个函数内部是跟据不同的条件去检测是否需要对 PWM 做清除的，实际上执行清除的部分还是放在了 PID 函数里面的。

9. 设置 PWM 数值做电机控制 31:38-34:57

再往下就是根据不同的小车型号设置不同的 PWM 控制极性，这里使用 Set_Pwm 这个函数，以电机 A 为例来分析一下这个函数，假如电机 A 传进来的 PWM 数值是小于 0 的，那么就让 PWMA1 等于 16799，相当于这个 PWM 输出通道输出了高电平，而后面的 A2 则等于 16799 加上传进来的数值，因为是小于 0 的所以加上等于减去传进来的参数的绝对值，A2 的的值也就小于 16799，这样的话 A1 跟 A2 就形成了电压差，电机就可以转动起来；那如果这个值大于等于 0 的话，则跟前面是相反的的，A2 等于 16799，A1 等于 16799 减去传进来的 PWM 数值，这个时候 A1 跟 A2 也形成了电压差，但是这个电压差的极性跟上面的电压差极性相反，所以电机转动的方向也就相反了；其他三个电机也是这么控制的；最后是阿克曼小车中舵机的控制，只需要把舵机的 PWM 给 Servo_PWM 这个宏定义就可以了，那这个宏定义可以查看一下，是定时器 12 的通道 2，那在前面初始化的时候是有初始化了两个 PWM

引脚的，但是实际上使用的是时候是只使用了通道 2，对应的引脚是 PB15，也就是最靠近板子边沿的舵机接口，前面讲原理图的时候也有提到这是为了初始化板子的所有资源，为了方便理解，**后续优化代码的时候会考虑将没使用到的引脚初始化去掉**。

那前面说的都是电池电压、使能开关以及软件停止位都正常的情况下执行的代码，如果检测到有异常的话，Servo_PWM 函数之后传入五个 0，这个时候电机跟舵机均处于静止，小车也不允许控制。

10. 写 Flash34:57-35:47

在控制任务的最后是 Flash 的写入，进入函数可以看到这里写入的实际就是四个变量，那在前面讲初始化的时候有看到一个读 Flash 的函数，那个函数也是读取这四个变量，那对应的写 Flash 就是现在看到的这个函数，这里是写入了控制时小车的速度、PID 控制的 P 值、PID 控制的 I 值以及纠偏系数，因为 Flash 在断电之后内容不会被擦除，所以这里写入的数据在后续重新上电之后依旧能被读取到。